Universidade Federal de Mato Grosso
Disciplina: Estrutura de Dados 1 Docente: Dr. Ivairton
Discente: Humberto Henrique Castro Rocha Matrícula: 202512722003

Relatório — *04 de março de 2026*

Análise de Ordenação de Dados de Filmes

Introdução

Este relatório apresenta a implementação e a análise de desempenho de três algoritmos clássicos de ordenação: *Quick Sort*, *Heap Sort* e *Merge Sort*. O objetivo principal foi aplicar esses métodos em um cenário real, utilizando um subconjunto de dados da base *MovieLens*, contendo títulos, anos de lançamento e avaliações.

Metodologia e Implementação

A base de dados original foi tratada para gerar um arquivo unificado em formato `.csv`, contendo os campos obrigatórios: Título, Ano, Gênero, Diretor e Avaliação. A leitura foi feita em C utilizando alocação dinâmica e a função `sscanf` com delimitadores de ponto e vírgula.

Os algoritmos foram implementados seguindo as definições clássicas: o *Quick Sort* por partição, o *Heap Sort* por construção de árvore binária completa em array, e o *Merge Sort* por divisão e intercalação.

Resultados e Discussão

Os testes foram realizados com um subconjunto de **435 filmes**. O critério utilizado foi a ordenação por **Avaliação (Rating)**. Os tempos obtidos através da biblioteca `time.h`

foram:

Tabela 1: Tempo de execução dos algoritmos (435 registros)

Algoritmo	**Tempo de Execução (segundos)**
Heap Sort	**0,001044**
Merge Sort	0,001375
Quick Sort	0,002225

Neste cenário específico, o **Heap Sort** apresentou o melhor desempenho. Este resultado é tecnicamente relevante, pois mostra que para conjuntos de dados de tamanho moderado, a eficiência do *Heap Sort* em manipular o array *in-place* sem o custo de múltiplas chamadas recursivas pesadas (como no Quick Sort) ou alocação de memória auxiliar (como no Merge Sort) pode resultar em um tempo de processamento menor.

O *Quick Sort*, embora teoricamente muito veloz, obteve o tempo mais elevado nesta amostra, o que pode ser atribuído ao custo de particionamento e à profundidade da recursão para uma lista de 435 elementos.

Conclusão

A análise experimental confirmou que todos os algoritmos testados operam dentro da complexidade esperada de $O(n \log n)$. A prática demonstrou que o *Heap Sort* foi a escolha mais eficiente para o volume de dados do projeto, unindo baixo consumo de memória com o menor tempo de execução. O trabalho cumpriu o objetivo de aplicar estruturas de dados de ordenação em um contexto de engenharia de software real.